

Python 数据分析与机器学习核心知识点总结

Python数据分析与机器学习核心知识点总结

一、数据预处理与工具库基础

1. pandas: `value_counts` 方法

- 作用：统计序列中各唯一值的出现次数。
- 注意事项：`pd.value_counts(y)` 已被弃用，推荐使用 `pd.Series(y).value_counts()` 避免 `FutureWarning`。

2. Matplotlib: `legend` 图例

- 作用：为图表添加图例，标识不同数据系列的含义。
- 用法：绘制图形时通过 `label` 参数指定系列名称，再调用 `plt.legend()` 生成图例；也可手动传入图形句柄和标签，如 `plt.legend([handle1, handle2], ["label1", "label2"])`。
- 常用参数：`loc`（指定图例位置，如 `'upper right'`）、`title`（设置图例标题）等。

3. 数据标准化与归一化

- 背景：KMeans等依赖“距离”的算法对特征“尺度”敏感（如一个特征范围 `0-1000`、另一个 `0-1` 时，距离计算会被大尺度特征主导）。
- 方法选择：
 - **标准化（`StandardScaler`）**：将特征转换为“均值=0，方差=1”的分布，公式为 (
$$x_{\text{新}} = \frac{x - \text{均值}}{\text{标准差}}$$
)。适合数据存在异常值、分布接近正态的场景。

- **归一化 (MinMaxScaler)**：将特征压缩到 `[0,1]` 区间，公式为 $(x_{\text{新}} = \frac{x - \text{最小值}}{\text{最大值} - \text{最小值}})$ 。适合数据无异常值、需要固定范围（如输入到神经网络）的场景。
- 注意：只需选一种方法，无需同时使用；训练集用 `fit_transform`，测试集用 `transform`（避免“数据泄露”）。

二、聚类算法

1. KMeans聚类

- 核心逻辑：通过“样本到聚类中心的距离”，将数据分为指定数量（`n_clusters`）的簇。
- 关键代码：

Code block

```
1 from sklearn.cluster import KMeans
2 KM = KMeans(n_clusters=3, random_state=0) # 初始化（指定簇数、固定随机种子保证结果可复现）
3 KM.fit(x) # 拟合数据
```

- 重要属性：
 - `KM.labels_`：每个样本的聚类标签。
 - `KM.cluster_centers_`：各簇的中心坐标。
- 注意事项：Windows+MKL环境下可能存在内存泄漏，需在Python进程启动前设置环境变量 `OMP_NUM_THREADS=6`（如通过命令行先设环境变量，再启动Python）。

2. MeanShift聚类

- 核心逻辑：基于“密度”的聚类算法，样本向“密度更高的区域”漂移，自动收敛到聚类中心（无需提前指定簇数）。
- 关键代码：

Code block

```
1 from sklearn.cluster import MeanShift, estimate_bandwidth
```

```
2  bw = estimate_bandwidth(x, n_samples=500) # 自动估计“带宽”（控制密度计算的范
    围)
3  ms = MeanShift(bandwidth=bw)
4  ms.fit(x) # 拟合数据
```

- 获取簇数量：MeanShift 无直接的 n_clusters 属性，需通过 len(np.unique(ms.labels_)) 统计“唯一聚类标签”的数量。

三、分类算法：KNN（K近邻）

- 核心逻辑：“物以类聚”——新样本的类别由其“最近的 k 个邻居”的多数类别决定。
- 关键代码：

Code block

```
1  from sklearn.neighbors import KNeighborsClassifier
2  KNN = KNeighborsClassifier(n_neighbors=3) # 初始化（指定邻居数 `k=3`）
3  KNN.fit(x, y) # 用特征 `x` 和标签 `y` 训练（存储训练数据，预测时实时计算距离）
```

- 特点：属于“惰性学习算法”，训练时不计算参数，仅存储数据；预测时才实时计算样本间距离。

四、其他关键概念

1. 内存泄漏

- 定义：程序申请内存后未正常释放，导致可用内存逐渐耗尽，最终使程序运行变慢甚至崩溃。
- 示例场景：KMeans在Windows+MKL环境下，因多线程与数据分块不匹配可能引发内存泄漏，需通过设置 OMP_NUM_THREADS 规避。

2. fit、transform、fit_transform

- fit：学习数据的统计规律（如标准化的“均值、标准差”，归一化的“最大/最小值”），仅学习、不改变数据。

- `transform`：用 `fit` 学到的规律转换数据。
- `fit_transform`：合并 `fit` 和 `transform`，一步完成“学习+转换”，适合**训练集**处理；测试集需用训练集的 `fit` 结果做 `transform`（避免“数据泄露”）。

以上为本次内容的核心知识点总结，涵盖数据预处理、聚类、分类算法及工具使用细节，可作为Python数据分析与机器学习实践的参考。

（注：文档部分内容可能由 AI 生成）